

REPORTE TAREA 1

ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Matias Huiscan Trincado

8 de septiembre de 2026

16:30

1. Introducción

En este laboratorio nos propusimos a medir, anotar y analizar distintos arreglos unidimensionales como fueron MergeSort, PatienceSort, QuickSort y `std::sort`, además de 2 algoritmos de multiplicación de matrices cuadradas que fueron Naive y Strassen. Con los datos obtenidos al correr distintas muestras pudimos ver el valor temporal de cada uno de estos algoritmos y ver si su comportamiento cuadra con su valor de complejidad teórico.

2. Entorno experimental

Las pruebas y mediciones de estos algoritmos fueron realizadas en un computador portátil con un procesador AMD Ryzen 3 7320U y 8 GB de memoria RAM DDR5. El entorno de ejecución utilizado fue Windows con el subsistema de Linux (WSL), y el código fue compilado utilizando g++ con la bandera de optimización `-O3`. Los datasets utilizados para las pruebas corresponden a los arreglos y matrices generados según los parámetros del enunciado mediante los scripts de Python.

3. Resultados y Análisis

3.1. Ordenamiento de Arreglos

Como podemos ver en ambos gráficos (ver Figuras 1 y 2), MergeSort con su complejidad teórica de $O(n \log n)$ en todos los casos tiene un tiempo constante debido a su estrategia de dividir y conquistar,

ademas en el grafico de barras se puede ver que sin importar como venga el arreglo tiene un tiempo promedio con respecto a los demas.

QuickSort tiene un caso promedio de $O(n \log n)$, pero su analisis del peor caso cae a $O(n^2)$ cosa que se demuestra en los graficos: teniendo una demora baja en comparacion a los demas, esta se ve reducida enormemente debido a la distribucion de los arreglos, ya que su pivote queda desbalanceado.

PatienceSort su complejidad teorica si es que se optimiza bien puede llegar a ser de $O(n \log n)$, pero debido a que en el codigo que use viene con implementacion de codigos dinamicos y una funcion `v.erase()` que desplaza la memoria en cada extraccion, lleva a este algoritmo a una complejidad ineficiente de $O(n^2)$.

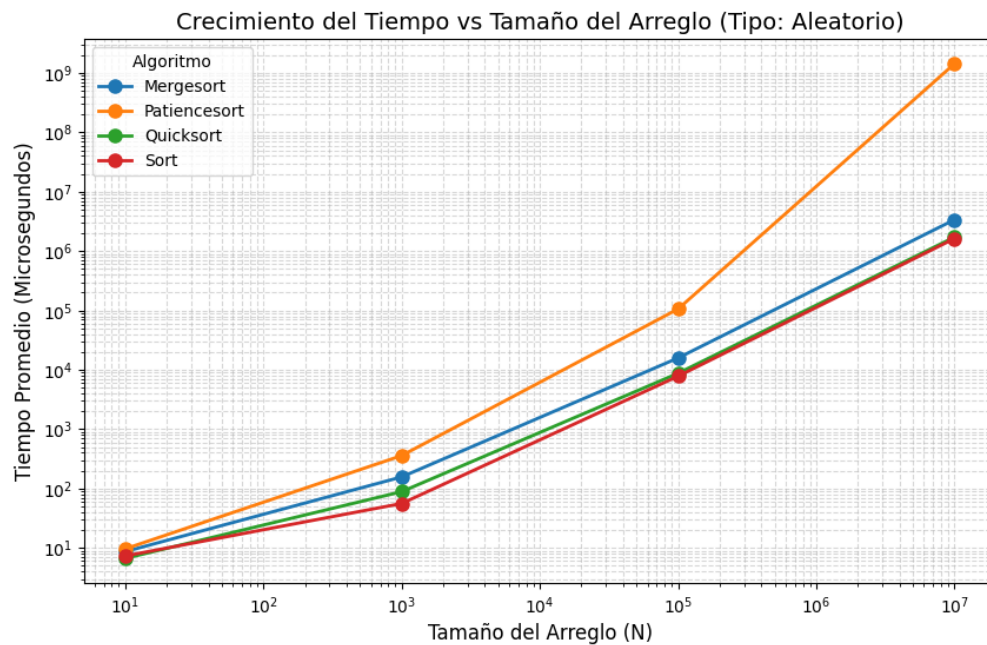


Figura 1: Crecimiento del tiempo segun tamaño N.

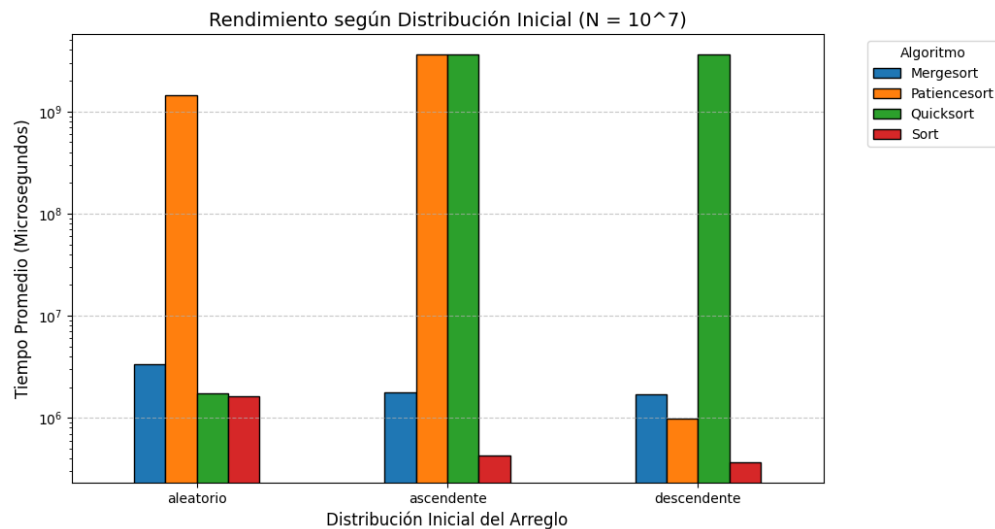


Figura 2: Rendimiento de ordenamiento según distribución.

3.2. Multiplicación de Matrices

Por otro lado los algoritmos de multiplicación de matrices cuadradas (ver Figuras 3 y 4) como el algoritmo Naive que tiene una complejidad teórica de $O(n^3)$ ya que tiene 3 ciclos anidados de tamaño n se demuestra en los gráficos de tiempo ya que estos van subiendo a medida que se va agrandando la matriz.

Por último el algoritmo de Strassen que tiene una complejidad asintótica de $O(n^{2.81})$ pero debido a como se implementa en el código que use, que hace muchas sumas de matrices y llamadas recursivas, hace que este suba su tiempo viéndose más lento en el gráfico de tiempo en comparación al Naive.

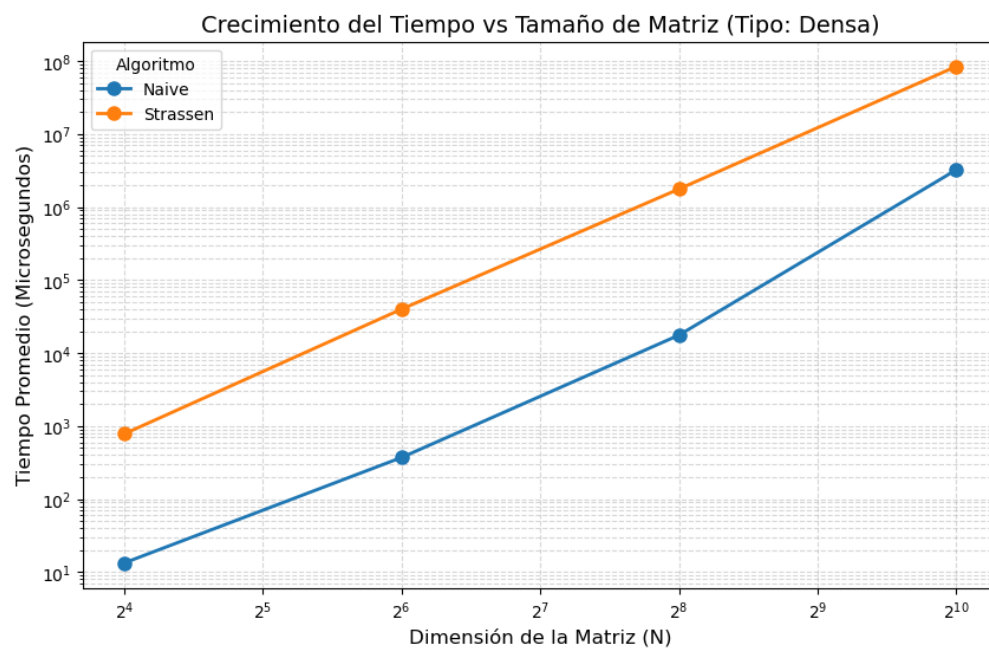


Figura 3: Crecimiento del tiempo en matrices densas.

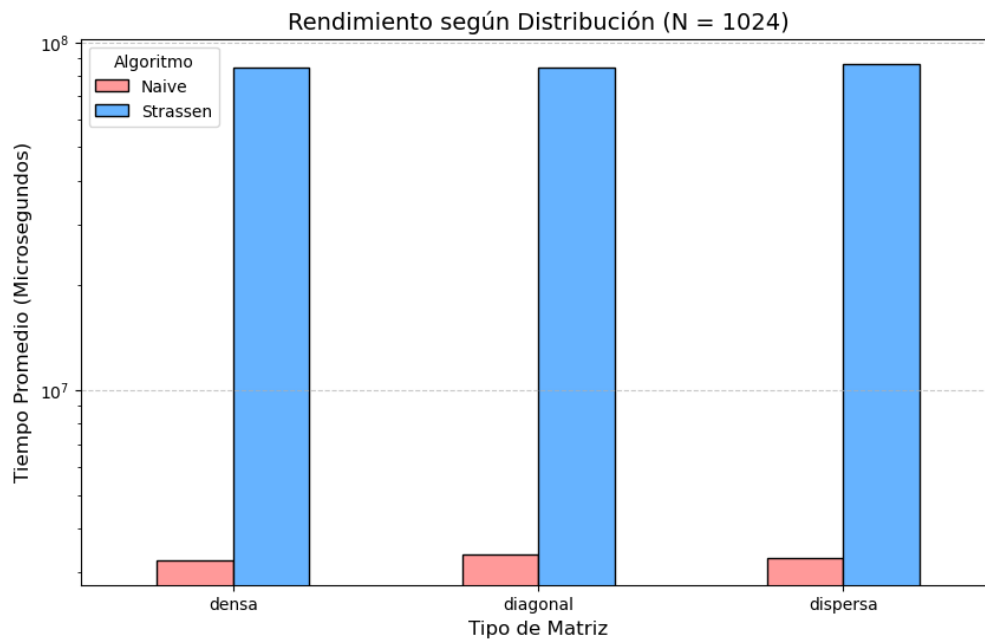


Figura 4: Rendimiento en matrices segun tipo.

4. Conclusiones

Con todos los datos recopilados y sabiendo que para los algoritmos de ordenamiento de arreglos los deje con un tope de 3600 segundos para decidir el peor no queda claro tomando en cuenta que tanto PatienceSort en general o como QuickSort cuando los arreglos estan ordenados dan los tiempos tope cuando los arreglos son muy grandes. Lo que si podemos hacer es ver el mejor algoritmo que segun los graficos de tiempo para arreglos grandes es el `std::sort` ya que domina en las 3 distribuciones, aunque si lo vemos para arreglos pequeños el ganador es sin duda QuickSort ya que su tiempo promedio es menor en arreglos mas pequeños.

Por otro lado al comparar los algoritmos de multiplicacion de matrices nos quedan 2 cosas segun los graficos: la primera es la superioridad del algoritmo Naive en matrices pequeñas ya que como podemos ver en el grafico de tiempo este se iba haciendo mas grande mientras mas grandes sean las matrices. Mientras que para el algoritmo Strassen, por el costo de sus operaciones, va a llegar un momento donde nos es mas optimo utilizar este algoritmo solo cuando las matrices sean muy grandes.

Referencias

- [1] Equipo de Algoritmos y Complejidad UTFSM. *Clases de Multiplicación de matrices: algoritmo cúbico, localidad de memoria, D&C ingenio, Strassen - Teorema Maestro*. Material de clases INF-221. 2026.
- [2] Thomas H. Cormen et al. *Introduction to Algorithms*. 3rd. Cambridge, MA: MIT Press, 2009.
- [3] Jeff Erickson. *Algorithms*. Jun. de 2019. ISBN: 978-1-792-64483-2.

- [4] Christophe Fiorio. *Algorithm2e documentation*. <http://ctan.math.illinois.edu/macros/latex/contrib/algorithm2e/doc/algorithm2e.pdf>. 2023.
- [5] Christophe Fiorio. *Algorithm2e on CTAN*. <https://ctan.org/pkg/algorithm2e>. 2023.
- [6] GeeksforGeeks. *Easy Way to Remember Strassen's Matrix Equation*. <https://www.geeksforgeeks.org/dsa/easy-way-remember-strassens-matrix-equation/>. 2026.
- [7] GeeksforGeeks. *Patience Sorting*. <https://www.geeksforgeeks.org/dsa/patience-sorting>. 2026.
- [8] David Hales. «An Introduction to Replicability in Computer Science Research». En: *Journal of Computational Methods* 13.1 (2017), págs. 123-145.
- [9] Donald E. Knuth. *The art of computer programming, volume 3: (2nd ed.) sorting and searching*. USA: Addison Wesley Longman Publishing Co., Inc., 1998. ISBN: 0201896850.
- [10] Overleaf. *Writing Algorithms in LaTeX*. <https://www.overleaf.com/learn/latex/Algorithms>. 2023.
- [11] Programiz. *Merge Sort Algorithm*. <https://www.programiz.com/dsa/merge-sort>. 2026.
- [12] Programiz. *Quick Sort Algorithm*. <https://www.programiz.com/dsa/quick-sort>. 2026.